



Sede Regional
Río Tercero
**ESCUELA SUPERIOR
de COMERCIO**

UPC UNIVERSIDAD
PROVINCIAL
DE CÓRDOBA

Programación Orientada a Objetos



Mignani Agustina

Universidad Provincial de Córdoba

Programación Full Stack - 2 año

Guerra Mario

28 de mayo, 2026

Diagnóstico de Diseño: Cohesión y Acoplamiento

Nivel de Cohesión (Principio de Responsabilidad Única - SRP)

Al analizar la estructura de la clase Curso, identifiqué que el método de inscripción original concentraba múltiples subtareas dentro de su flujo principal. La cohesión mide la conectividad de los elementos internos de un módulo, y la buena práctica exige buscar una alta cohesión donde cada clase se enfoque en una sola tarea clara. El diseño original mezclaba la validación de datos, el control de vacantes y la persistencia en una sola rutina.

Para resolverlo y aplicar el principio DRY (Don't Repeat Yourself), refactoricé el comportamiento dividiendo el flujo en dos métodos privados que expresan su intención desde su nombre:

- `estaInscripto(int dni)`: Dedicado exclusivamente a rastrear duplicados recorriendo el `ArrayList` mediante la API de Streams.
- `procesarInscripcion(Alumno alumno)`: Centraliza únicamente la regla del cupo máximo y el alta en la lista.

Con esta modularización, si el día de mañana la institución cambia la forma de validar a los alumnos o modifica las penalizaciones de cupo, solo tenemos que tocar un método atómico específico, sin romper el resto de la clase.

Nivel de Acoplamiento y Encapsulamiento

Un punto crítico que detecté está en la visibilidad de los atributos de la clase abstracta Persona. Había optado por usar el modificador `protected`. Aunque inicialmente se plantea para que Alumno y Profesor hereden el acceso directo, la teoría explica que el acoplamiento mide la dependencia entre clases y que se debe perseguir un bajo acoplamiento para que los cambios internos no fuercen la modificación de otros componentes. Dejar los campos desprotegidos genera un acoplamiento estructural entre la superclase y sus herederas.

Propuesta de refactorización: Para respetar el encapsulamiento estricto y lograr el correcto ocultamiento de la información, cambie la visibilidad de los atributos a `private`. Esto protege el estado interno del objeto y obliga a las subclasses a comunicarse con el padre mediante el "paso de mensajes" a través de los getters y setters públicos. Así se aísla la estructura de datos y se facilita cualquier cambio futuro en la API interna de la clase sin romper sus dependencias.

Matriz de Análisis de Buenas Prácticas y Refactorización

Criterio Evaluado	Estado Detectado en el Código	Solución / Sugerencia de Refactorización
<i>Nombres y Nomenclatura</i>	Use variables de una sola letra (ins -> ...) o bucles (i).	Mejore la legibilidad reemplazándolos por nombres con significado semántico (inscripcion, persona). Esto hace que el código sea autodocumentado y no requiera tantos comentarios extras.
<i>Organización y Flujo</i>	Use lógica condicional anidada tradicional dentro del algoritmo de inscripciones rápidas.	Al validar los errores al principio del método y salir inmediatamente con un return, limpie los bloques else y el código queda plano y directo.

<i>Reutilización</i>	Había duplicidad de estructuras de formato e impresiones por pantalla directamente quemadas en consola.	Resolvi implementando el contrato puro de la interfaz Reportable. Ahora, cada objeto sabe exactamente cómo formatear su salida mediante obtenerReporteLimpio(), centralizando el formato.
<i>Extensibilidad</i>	Dependencia directa de la clase concreta Alumno en los flujos internos del módulo Curso.	Al acoplar el bucle final del Main a la abstracción de la interfaz Reportable, el sistema queda abierto a la extensión. Si en el futuro agregamos clases como PersonalAdministrativo, se listarán automáticamente sin modificar el motor del programa.

Evaluación del Impacto del Polimorfismo Aplicado

Polimorfismo Estático (Sobrecarga)

En la clase Curso, el método inscribirAlumno ofrece flexibilidad al usuario de la aplicación. Puede realizar una inscripción formal enviando la instancia completa del Alumno con todos sus datos o una inscripción rápida usando parámetros primitivos, resolviéndose en tiempo de compilación según los argumentos provistos.

Polimorfismo Dinámico (Sobrescritura)

Mediante el método abstracto mostrarRol(), la Máquina Virtual de Java resuelve en tiempo de ejecución qué comportamiento ejecutar basándose en el tipo real del objeto en memoria, permitiéndonos iterar de forma transparente sobre la lista de tipo Persona.

Polimorfismo por Interfaz

A través de este enfoque, logre unificar dentro de una misma colección (ArrayList<Reportable>) objetos de familias completamente diferentes, como una entidad de la jerarquía de herencia (Persona) y un objeto de asociación pura (Inscripcion). Esto demuestra cómo las interfaces nos permiten crear contratos de comportamiento totalmente independientes de la herencia estructural.